

Unit 1: Introduction to OOP & C++.

* ^(POP) ~~Diff~~ Procedure oriented programming & Object oriented programming (OOP)

POP	OOP.
① In POP, program is divided into small parts called 'functions'.	① In OOP, program is divided into small parts called 'Object'.
② In POP, importance is given to sequence of actions, not the data.	② In OOP, importance is given to data, rather than the procedure.
③ POP follows top-down approach approach.	③ OOP follows bottom-up approach.
④ In POP, does not have any proper way for hiding data, so it is less secure.	④ OOP, provides data hiding feature, so it is more secure.
⑤ In procedure POP, overloading is not possible.	⑤ In OOP, overloading is possible / done. to ex. function overloading overloading, and operator overloading.
⑥ ^{ex.} C, VB, FORTRAN, PASCAL.	⑥ C++, JAVA, VB.NET, Python, JS, C#.

* characteristics of C++ language.

① Object:

Object is an runtime entity, in an object oriented system, it may represent a place, person, bank etc.

②

Object may also represent, user defined data such as Vectors, lines, time.

②

Class:

A class in C++, is a user defined data structure declared with keyword class.

Syntax:

~~class~~ ~~class-name~~

Every class has data members and member functions as its members.

③

Data encapsulation:

Wrapping up of data and functions into a single unit is known as encapsulation, means, the data is not accessible to the outside world, whereas but accessible to function which are wrapped in ~~class~~ class can access it.

④

Data Abstraction.

④ Data Abstraction: (Data hiding)

Showing essential details and hiding other data.

⑤ Inheritance:

Inheritance is the process by which, ~~one~~ one class acquires properties of another class.

⑥ Polymorphism:

Polymorphism is the ability to take more than one form of function, class, method.

Types of polymorphism:

- ① compile-time-polymorphism.
- ② Run-time polymorphism.

⑦ Dynamic Binding:

The implementation of the function that is used on an particular call is determined dynamically at run-time. This capacity ~~at run-time~~ is known as ~~dynamic~~ Dynamic binding.

Dynamic Binding is the ~~relation~~ connection between function declaration and function call.

~~⑧~~ ✓

⑧ message-passing:

(8) Virtual function:

Virtual function is a member function, declared in a base class and re-declared in a derived class.

* Benefits of ~~CPP~~ OOP

- ① OOP provides reusability to the code/data
- ② ~~Data~~ OOP helps in data hiding, keeping the data & information safe from getting exposed.
- ③ OOP is very easy to ~~obtain~~ implement/maintain.
- ④ Programming is more readable and having better performance in OOP.
- ⑤ OOP is a open source language program and has benefits like code reusability, data encapsulation, abstraction, inheritance which makes data modular.

* ~~Structure of C++~~

* Concepts related to OOPs

① variable: variables are the backbone of any programming language. It is a way to store information for later use. we can retrieve the value ^{or} data from a variable, by referring to the particular address.

Once the variables are declared and defined, they may be used many times in a program, within the scope of the code.

② Control Structures:

When the code is being read from top to bottom, it may encounter a point, where it needs to make a decision. Based on that decision, the program may jump to different part of code.

③ C++ Data Structures:

④ Structure of C++

```
#include <iostream>
// using namespace std;
int main ()
{
    cout << "welcome to 8XIT"; // O/P
    return 0;
}
```

• This `<iostream>` header file contains the declaration for `cout` and `cin` header operators.

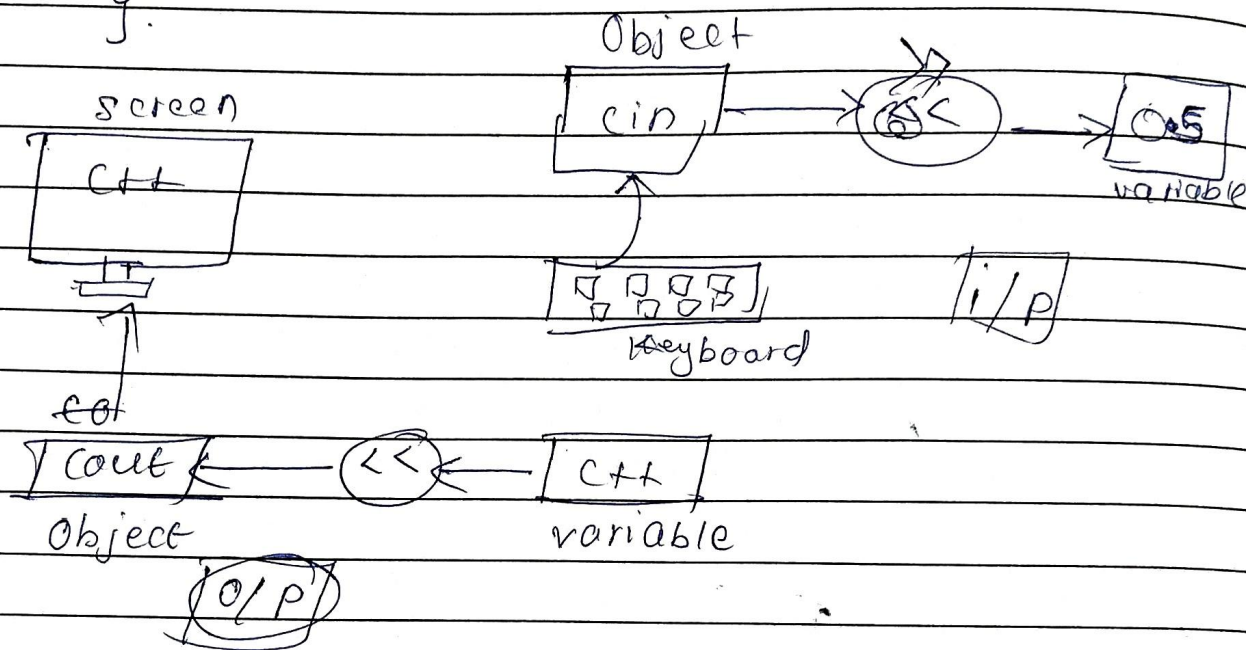
This directive causes the preprocessor to add the contents of `iostream` to the program.

* write a c++ program to add two numbers.

```
#include <iostream>
using namespace std;
int main ()
{
    int A, num1, num2;
```

```
cout << "Enter two numbers ";
cin >> num1 >> num2;
```

```
cout << "The sum is: " << num1 + num2;
return 0;
}
```



* ~~Retraction operator~~ Insertion operator

The identifier `cout` is a predefined object that represents the standard output stream in C++.

The operator '<<' is called as the insertion operator or 'put to' operator.

It inserts the content of variable on its right to the object on its left.

* Extraction operator (>>)

The identifier `cin` is a predefined object that represents standard input stream in C++.

The operator ">>" is called extraction operator or "get from".

It extracts/takes value from keyboard and assigns it to variable on right.

* Structure of C++ program

① #include <iostream>

This directive causes the preprocessor to add the contents of the `iostream` file to the program.

This header file contains the declaration for `cin`, `cout` identifier and ~~extra~~ `extract`.

This file is always written at start of C++ program.

② using namespace std;

Namespace is the new concept, introduced by ANSI, C++ standards. This defines a scope for the identifiers, that are used in the program. For using the identifiers defined in the namespace scope, we must include this ~~directly~~ directory.

`std` is the namespace, where ~~ANSI~~ C++, standard class libraries are defined.

~~***~~

Token

The smallest individual unit in programming are known as tokens.

A token is the smallest element of a program, that is meaningful to the compiler.

eg. C++ tokens are as follows:-

- (1) keyword
- (2) Identifier
- (3) constants
- (4) string
- (5) operator
- (6) special symbols

(1) keyword: ~~keywo~~

Keywords are the identifiers, which are reserved explicitly, ~~mean~~ and cannot be used as names for programming variables.

(2) Identifier:

~~Ref~~ Identifier refers to name of variable, function, array, classes ~~which are~~ etc created by the programmer.

They are fundamental requirements of any programming language.

Rules

- (1) Only alphabetic characters, digits and underscore are permitted.
- (2) These identifier names cannot be started with digit or underscore ~~or any special~~
- (3) Lowercase and uppercase letters are distinct
a declared keyword cannot be used as a variable name.

③ constants:

constants refer to the fixed values that do not change during the execution of the program.

C++ supports several kinds of literal constants for eg. integer constant, character constant, floating point numbers and string constant.

④ String:

Strings are nothing but the array of characters ended with null character (`'\0'`), strings are always enclosed in double quotes (""), whereas char are enclosed in single quotes.

```
# char str[10] = { 'S', 'Y', 'I', 'T', 'B', 'D',  
                  'I', 'V', '\0' };
```

```
char str[10] = "SYITB DHRCEM";
```

⑤ Operator:-

C++ has a rich set of operators and all C operators are valid in C++.

Operator

Definition:- "Operator are symbols that trigger an action when applied to C++ variables and other objects."

The data items on which operators act upon are called as 'operands'.

Types of Operators

① Depending on number of operands, operators have three types

① unary operator ② Binary operator ③ Ternary operator

① unary operator: Operator that works on only one operand are called unary operator
eg. +, -, !

② Binary operator: operator that works on two operands are called binary operator

③ eg. Arithmetic - +, -, *, \

logical - &, ||, !

Relational: ==, <=, >=, >, <

Assignment: (=)

Bitwise operator: &, |, ^, <<, >>

insertion: <<

extraction: >>

scope resolution: ::

~~operator~~

pointer to member declaration

Operator ::*

→ *, .* = pointer to member operator

delete = releases memory allocated to variable, function object etc

(memory allocation operator) new operator = assigns memory

(line feed operator) endl = new line

(field width operator) setw = specifies the field width of variable

(conditional operator)

③ Ternary Operator: Operator ~~works~~ works on more than two operands. (~~if~~ $?:$)

④ Function Overloading:

Q. what is

→ It is a feature of OOP, where two or more function can have the same name, but different parameters. When a function, ~~a function~~ is overloaded with different functionality, then it is called as function overloading.

```
#include <iostream>
```

```
using namespace std;
```

```
void add(int a, int b)
```

```
{ cout << "sum" << (a+b);
```

```
}
```

```
void add(int a, int b, int c)
```

```
{ cout << "sum" << (a+b+c);
```

```
}
```

```
int main()
```

```
{ int a, b, c = 5, b = 7, c = 8;
```

```
  add(a, b); // call
```

```
  add(4, 8)
```

```
  add(a, b, c); // call
```

```
  add(5, 5, 6)
```

```
  return 0;
```

```
}
```

~~ii) Data Data types are~~

~~ii)~~

* operator overloading :

c++ has the ability to provide the operators with a special meaning, without changing its original meaning.

* Difference between constructor & Destructor in C++.

Constructor	Destructor
① Constructor helps to initialize object of class	① Destructor helps to destroy objects
② Constructor can either accept arguments or not.	② Destructors can't have any arguments.
③ A constructor is called when an object of a class is created.	③ Destructor is called when the object of a class is deleted/ <u>released</u>
④ Constructor is used to allocate memory to object.	④ Destructor is used to deallocate memory to object.
⑤ Constructor can be overloaded	⑤ Destructor cannot be overloaded.

⑥ constructor name is same as that of class name.

⑥ ~~tot~~ destructor name is same as that of class name preceded by Tilde (~)

⑦ In a class there can be multiple constructor

⑦ In a single class there is always a single destructor.

⑧ There is a concept of copy constructor which is used to initialize an object from another object.

⑧ There is no concept as copy destructor

⑨ constructor is always called in successive order

⑨ destructor are called in reverse order of a constructor.

Syntax:

```
classname()
{
    //constructor body
}
```

Syntax:

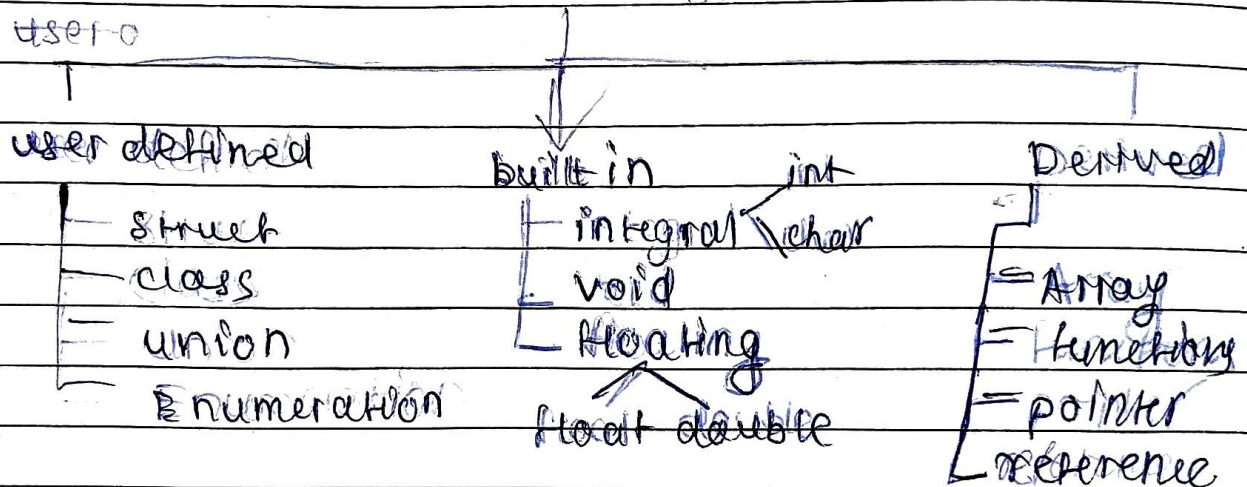
```
~classname()
{
    //destructor body
}
```

*

Data Type :-

A Data type is simply a type of attribute on data, which tells a compiler or interpreter how the programmer intends to use the data.

- ① user defined
- ② in-built data type
- ③ Derived data type.

C++ data type

*

void main()

:- specifies return type of main function

A

void main(void)

indicates empty argument list